



UNIVERSIDAD

Nicolás Felipe Bernal Gallo

Juan Daniel Bogotá Fuentes

Desarrollo Orientada a objetos
DOPO LAB

Laboratorio #4 Excepciones
23/10/2025

PROFESOR: María Irma Diaz Rozo

DESARROLLO ORIENTADO POR OBJETOS [DOPO-POOB]

Excepciones

2025-2

Laboratorio 4/6

OBJETIVOS

1. Perfeccionar el diseño y código de un proyecto considerando casos especiales y errores.
2. Construir clases de excepción encapsulando mensajes.
3. Manejar excepciones considerando los diferentes tipos.
4. Registrar la información de errores que debe conocer el equipo de desarrollo de una aplicación en producción.
5. Practicar las prácticas *Designing* - [Simplicity](#).

□ [Refactor](#) whenever and wherever possible.

ENTREGA

- Incluyan en un archivo **.zip** los archivos correspondientes al laboratorio. El nombre debe ser los dos apellidos de los miembros del equipo ordenados alfabéticamente.
- Deben publicar el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios preparados para tal fin.

CONTEXTO: Clash Royale

Es un videojuego de estrategia en línea basado en los personajes de Clash of Clans.

- Cada jugador tiene un mazo de 8 cartas que representan tropas, hechizos o estructuras.
- Las cartas se juegan usando elixir, un recurso que se regenera con el tiempo.
- El objetivo es destruir las torres del oponente, especialmente la torre central (la del rey).

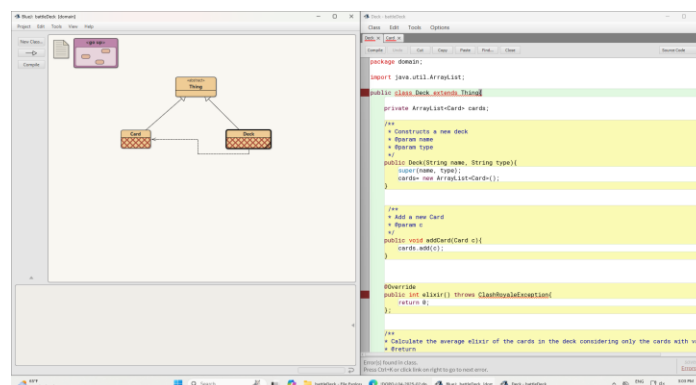
EN BLUEJ

PRACTICANDO MDD y BDD con EXCEPCIONES

[En lab04.doc, clashRoyale.asta y BlueJ units]

En este punto vamos a aprender a diseñar, codificar y probar usando excepciones. Para esto se van a trabajar algunos métodos de la clase **Deck**

1. En su directorio descarguen los archivos contenidos en [battleDeck.zip](#) revisen el contenido y estudien el diseño estructural de la aplicación (únicamente la zona en azul).



2. **Expliquen por qué el proyecto no compila. Realicen las adiciones necesarias para lograrlo.**

No compila porque se están propagando excepciones que no están definidas aun, tampoco está la clase de excepciones ClashRoyaleException.

```
ClashRoyaleException - battleDeck
Class Edit Tools Options
Deck X Card X ClashRoyaleException X
Compile Undo Cut Copy Paste Find... Close

package domain;

/**
 * Write a description of class ClashRoyaleException here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ClashRoyaleException extends Exception{

    public static final String ELIXIR_UNKNOWN = "No se conoce el elixir.";
    public static final String RESISTANCE_UNKNOWN = "No se conoce la resistencia";
    public static final String ELIXIR_ERROR = "El elixir no es suficiente";
    public static final String RESISTANCE_ERROR = "La resistencia no es suficiente";

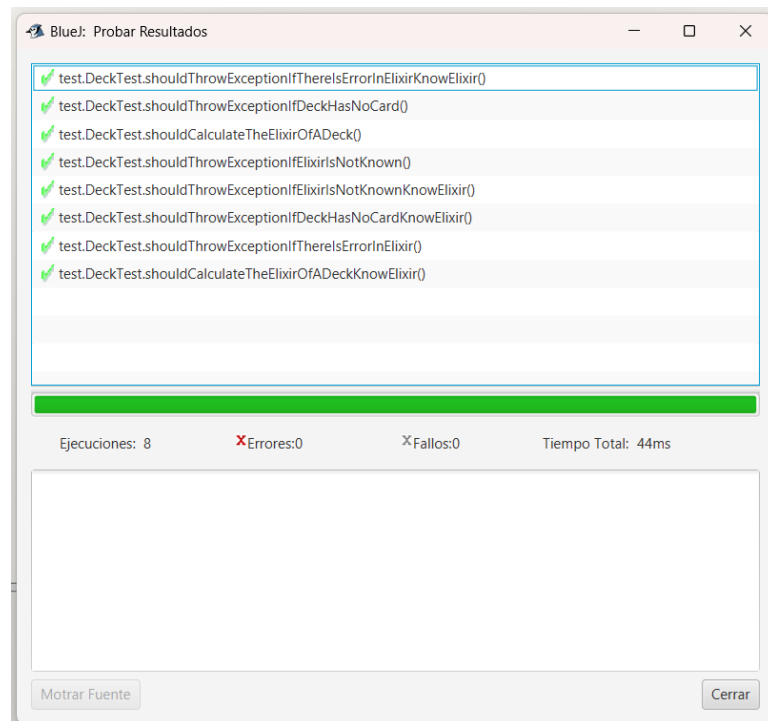
    public ClashRoyaleException(String message){
        super(message);
    }
}
```

3. **Dado el diseño y las pruebas documenten (si es necesario) y codifiquen el método elixir().**

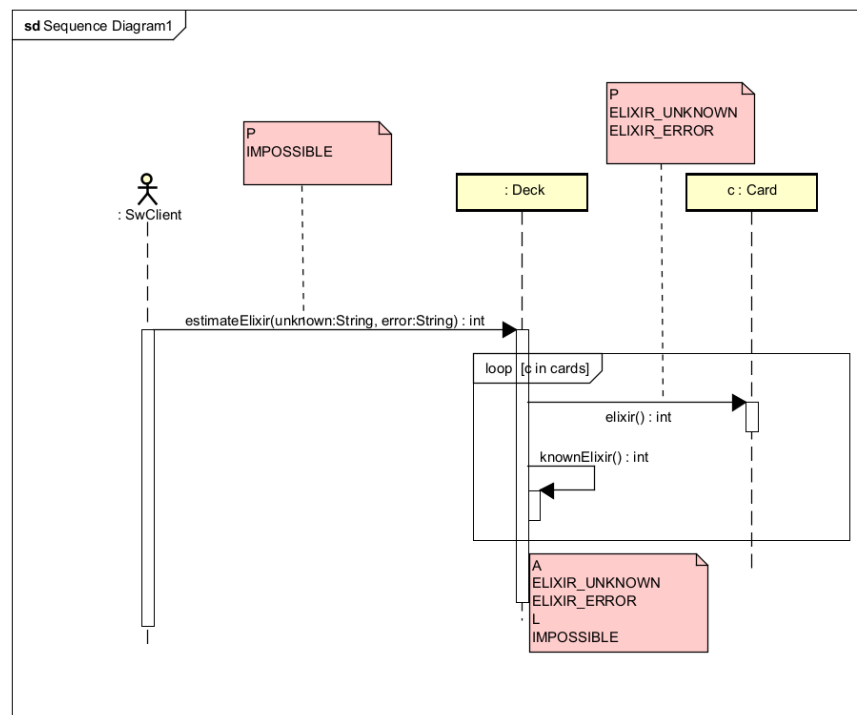
```
/**
 * Calculate the average elixir of the cards in the deck considering only the cards with valid values
 * @return the average
 * @throws ClashRoyaleException IMPOSSIBLE, If there are no valid values
 */
@Override
public int elixir() throws ClashRoyaleException{
    if (cards.isEmpty()) throw new ClashRoyaleException(ClashRoyaleException.IMPOSSIBLE);
    int sum = 0;
    for (Card c: cards){
        sum += c.elixir();
    }
    return Math.round((float) sum / cards.size());
}
```

4. **Dada la documentación y el diseño codifiquen y prueben el método knownElixir().**

```
/**
 * Calculate the average elixir of the cards in the deck considering only the cards with valid values
 * @return the average
 * @throws ClashRoyaleException IMPOSSIBLE, If there are no valid values
 */
public int knownElixir() throws ClashRoyaleException{
    if (cards.isEmpty()) throw new ClashRoyaleException(ClashRoyaleException.IMPOSSIBLE);
    int sum = 0;
    int count = 0;
    for (Card c: cards){
        try {
            int v = c.elixir();
            if (v >= 0) {
                sum += v;
                count++;
            }
        } catch (ClashRoyaleException e) {
            // Skip cards with unknown or invalid elixir
        }
    }
    if (count == 0) throw new ClashRoyaleException(ClashRoyaleException.IMPOSSIBLE);
    return sum / count;
}
```



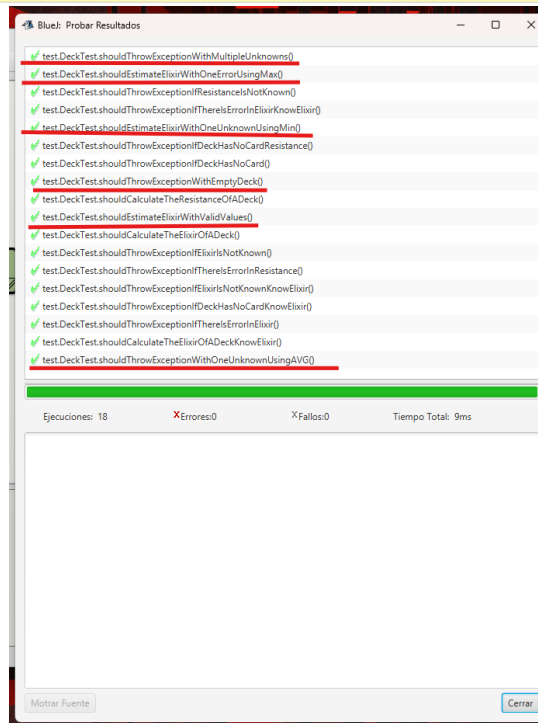
5. Documenten, diseñen, codifiquen y prueben el método `estimateElixir(unknown:String, error:String)` [unknown y error pueden tomar los siguientes valores MIN, MAX o AVG. Indican el valor que se usa para reemplazar un valor con error o desconocido: el mínimo, el máximo o el promedio de los valores válidos del mazo]



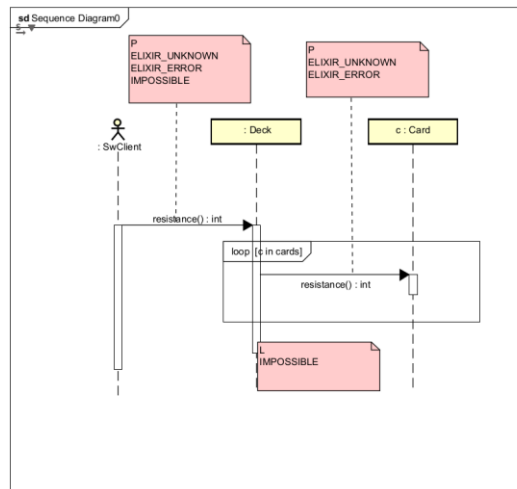
```

/**
 * Estimate the average elixir of the cards in the deck replacing unknown or
 * erroneous values
 *
 * @param unknown
 * @param error
 * @return the estimated average
 * @throws ClashRoyaleException
 */
public int estimateElixir(String unknown, String error) throws ClashRoyaleException {
    if (cards.isEmpty()) throw new ClashRoyaleException(ClashRoyaleException.IMPOSSIBLE);
    int validSum = 0;
    int validCount = 0;
    int unknownCount = 0;
    int errorCount = 0;
    for (Card c : cards) {
        try {
            int value = c.elixir();
            if (value >= 1 && value <= 9) {
                validSum += value;
                validCount++;
            } else {
                errorCount++;
            }
        } catch (ClashRoyaleException e) {
            unknownCount++;
        }
    }
    if (errorCount > 1 || unknownCount > 1)
        throw new ClashRoyaleException(ClashRoyaleException.IMPOSSIBLE);
    if (validCount == 0)
        throw new ClashRoyaleException(ClashRoyaleException.IMPOSSIBLE);
    int avg = 0;
    if ("AVG".equalsIgnoreCase(unknown) || "AVG".equalsIgnoreCase(error)) {
        avg = knownElixir();
    }
    int finalSum = validSum;
    if (unknownCount == 1) {
        finalSum += "MIN".equalsIgnoreCase(unknown) ? 1 : "MAX".equalsIgnoreCase(unknown) ? 9 : avg;
    }
    if (errorCount == 1) {
        finalSum += "MIN".equalsIgnoreCase(error) ? 1 : "MAX".equalsIgnoreCase(error) ? 9 : avg;
    }
    return finalSum / cards.size();
}

```



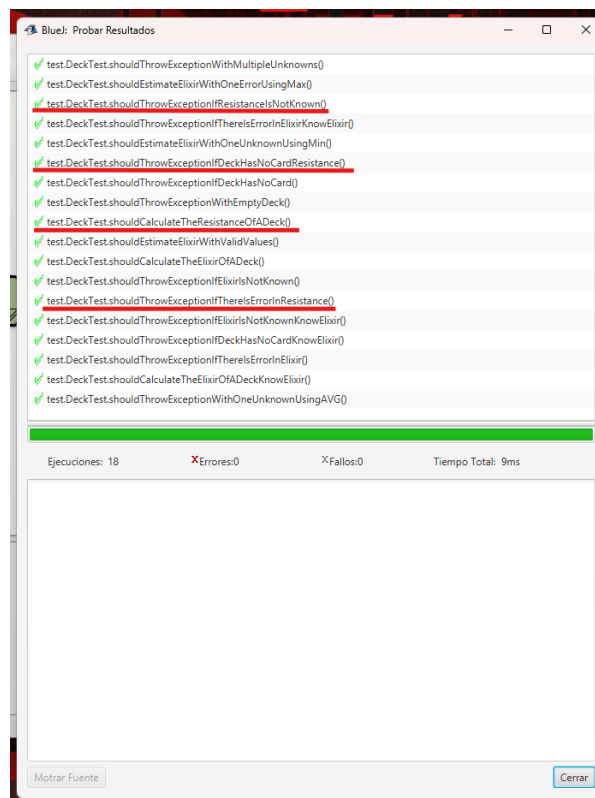
6. Propongan el cálculo, documenten, diseñen, codifiquen y prueben el método `resistance()`



```

/**
 * Calculate the average resistance of the cards in the deck considering only the cards with valid values
 * @return the average
 * @throws ClashRoyaleException IMPOSSIBLE, If there are no valid values
 */
@Override
public int resistance() throws ClashRoyaleException{
    if (cards.isEmpty()) throw new ClashRoyaleException(ClashRoyaleException.IMPOSSIBLE);
    int sum = 0;
    for (Card c : cards){
        sum += c.resistance();
    }
    return Math.round((float) sum / cards.size());
}

```



Clash Royale **EN CONSOLA**

El objetivo de esta aplicación es mantener un catálogo de mazos y cartas de *Clash Royale*.

Conociendo el proyecto ClashRoyale [En lab04.doc]

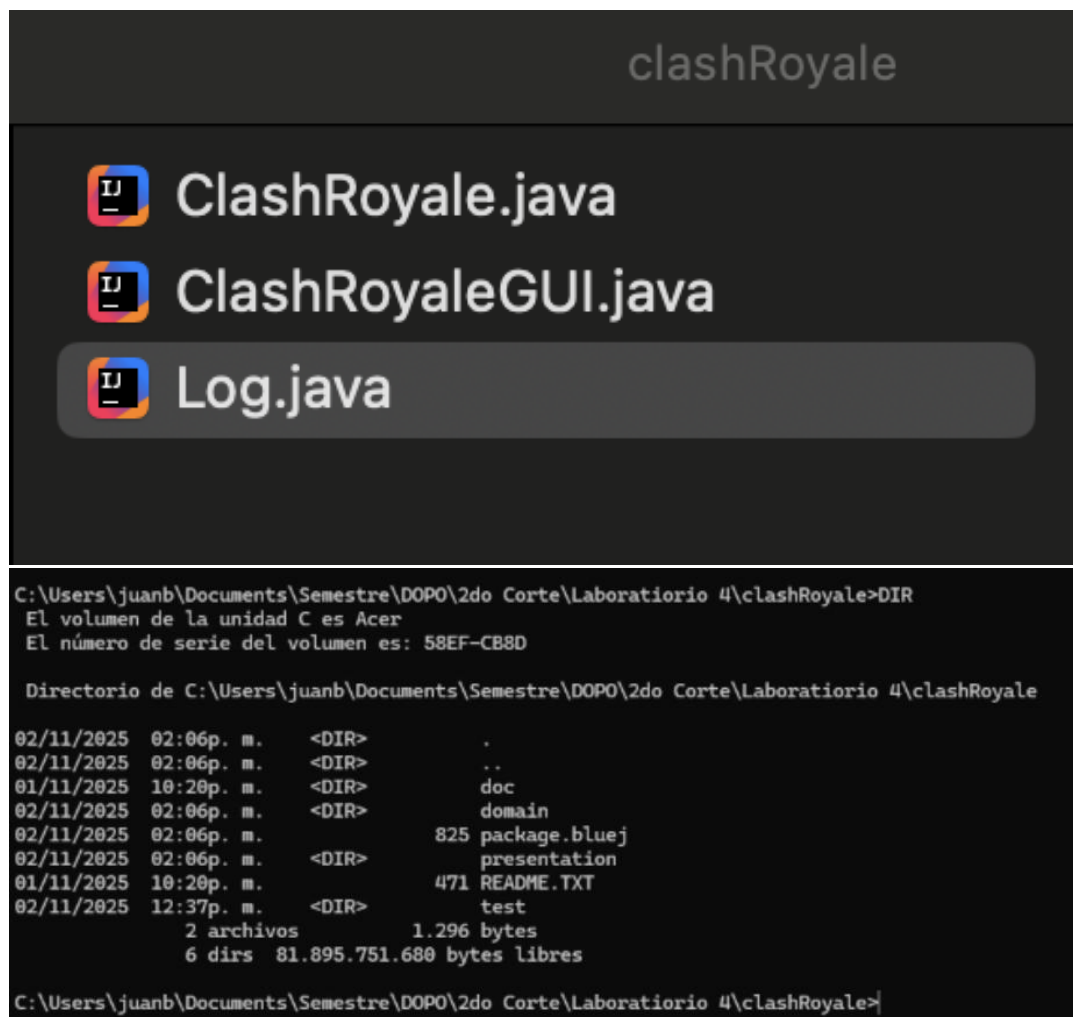
No olviden respetar los directorios bin docs src

1. En su directorio descarguen los archivos contenidos en [clashRoyale.zip](#), revisen el contenido.

¿Cuántos archivos se tienen?

Contiene 3 archivos como se puede ver en la imagen que son:

- ClashRoyale.java
- ClashRoyaleGUI.java
- Log.java (Que es donde quedan guardados los errores según lo explicado en clase por la profe).



```
clashRoyale

ClashRoyale.java
ClashRoyaleGUI.java
Log.java

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>DIR
El volumen de la unidad C es Acer
El número de serie del volumen es: 58EF-CB8D

Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale

02/11/2025  02:06p. m.    <DIR>        .
02/11/2025  02:06p. m.    <DIR>        ..
01/11/2025  10:20p. m.    <DIR>        doc
02/11/2025  02:06p. m.    <DIR>        domain
02/11/2025  02:06p. m.    825 package.bluej
02/11/2025  02:06p. m.    <DIR>        presentation
01/11/2025  10:20p. m.    471 README.TXT
02/11/2025  12:37p. m.    <DIR>        test
                2 archivos          1.296 bytes
                6 dirs  81.895.751.680 bytes libres

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>
```

```

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>DIR domain
El volumen de la unidad C es Acer
El número de serie del volumen es: 58EF-CB8D

Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale\domain

02/11/2025  02:06p. m.    <DIR>          .
02/11/2025  02:06p. m.    <DIR>          ..
01/11/2025  10:50p. m.      2.291 Card.class
01/11/2025  10:50p. m.      367 Card.ctxt
01/11/2025  10:46p. m.      1.363 Card.java
01/11/2025  10:20p. m.      3.588 ClashRoyale.java
01/11/2025  10:49p. m.      678 ClashRoyaleException.class
01/11/2025  10:49p. m.      407 ClashRoyaleException.ctxt
01/11/2025  10:20p. m.      771 ClashRoyaleException.java
02/11/2025  01:39p. m.      3.833 Deck.class
02/11/2025  01:39p. m.      1.717 Deck.ctxt
02/11/2025  01:38p. m.      4.522 Deck.java
01/11/2025  10:20p. m.      753 Log.java
02/11/2025  02:06p. m.      1.431 package.bluej
01/11/2025  10:49p. m.      689 Thing.class
01/11/2025  10:49p. m.      1.114 Thing.ctxt
01/11/2025  10:20p. m.      1.262 Thing.java
          15 archivos      24.786 bytes
           2 dirs      81.891.831.808 bytes libres

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>

```

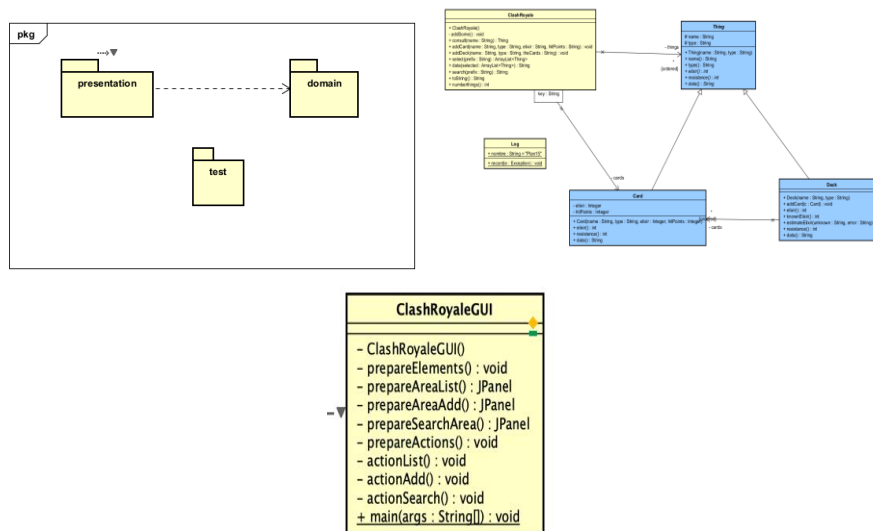
¿Cómo están organizados?

Todos los 3 archivos están en una misma carpeta.

¿Cómo deberían estar organizados?

Deberían estar organizados por paquetes/carpetas, la carpeta dominio: log.java y ClashRoyale.java y la carpeta presentación: ClashRoyaleGUI.java.

2. Estudien el diseño del programa: diagramas de paquetes y de clases.



¿cuántos paquetes tenemos?

Tenemos tres paquetes, presentation, test y domain, aunque no está el diagrama de paquetes, lo creamos.

¿cuántas clases tiene el sistema? ¿cómo están organizadas?

En el paquete de dominio tenemos 5 clases según el diagrama de clases, están organizadas por colores.

Además, las clases del paquete dominio maneja datos y reglas de negocio.

En el paquete de presentación tenemos solo una clase que es el ClashRoyaleGUI, en el paquete que muestra y recibe interacción del usuario.

```
C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>DIR presentation
El volumen de la unidad C es Acer
El número de serie del volumen es: 58EF-CB8D

Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale\presentation

02/11/2025  02:06p. m.  <DIR>      .
02/11/2025  02:06p. m.  <DIR>      ..
01/11/2025  10:20p. m.          7.281 ClashRoyaleGUI.java
                1 archivos          7.281 bytes
                2 dirs  81.891.516.416 bytes libres

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>DIR test
El volumen de la unidad C es Acer
El número de serie del volumen es: 58EF-CB8D

Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale\test

02/11/2025  12:37p. m.  <DIR>      .
02/11/2025  02:06p. m.  <DIR>      ..
02/11/2025  01:35p. m.          6.671 DeckTest.class
02/11/2025  01:35p. m.          1.606 DeckTest.ctxt
02/11/2025  02:05p. m.          8.621 DeckTest.java
02/11/2025  02:06p. m.          643 package.bluej
                4 archivos          17.541 bytes
                2 dirs  81.891.016.704 bytes libres

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>
```

¿cuál es la clase ejecutiva?

En la clase principal en la cual contiene el main() por el cual se va a ejecutar el programa.

3. Prepare los directorios necesarios para ejecutar el proyecto.

¿qué estructura debe tener?

Solo una carpeta llamada ClashRoyale, dentro hay tres paquetes, domain, presentation y test, sin contar la carpeta de documentaciones.

```
C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>DIR
El volumen de la unidad C es Acer
El número de serie del volumen es: 58EF-CB8D

Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale

02/11/2025  02:06p. m.  <DIR>      .
02/11/2025  02:06p. m.  <DIR>      ..
01/11/2025  10:20p. m.  <DIR>      doc
02/11/2025  02:06p. m.  <DIR>      domain
02/11/2025  02:06p. m.          825 package.bluej
02/11/2025  02:06p. m.  <DIR>      presentation
01/11/2025  10:20p. m.          471 README.TXT
02/11/2025  12:37p. m.  <DIR>      test
                2 archivos          1.296 bytes
                6 dirs  81.877.102.592 bytes libres

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\clashRoyale>
```

¿qué clases deben tener?

Debe tener: ClashRoyale, Thing, Card, Deck y Log, ClashRoyaleGUI y DeckTest.

¿dónde están esas clases?

El paquete domain debe tener: ClashRoyale, Thing, Card, Deck y Log.

El paquete presentation debe tener: ClashRoyaleGUI.

El paquete test debe tener: DeckTest

```
C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1>tree
Listado de rutas de carpetas para el volumen Acer
El número de serie del volumen es 58EF-CB8D
C:..
├── bin
│   ├── domain
│   ├── presentation
│   └── test
├── docs
│   └── doc
│       ├── domain
│       ├── presentation
│       └── test
└── src
    ├── domain
    ├── presentation
    └── test

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1>
```

```
C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1\bin\domain>dir
El volumen de la unidad C es Acer
El número de serie del volumen es: 58EF-CB8D

Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1\bin\domain

02/11/2025  02:58p. m.  <DIR>          .
02/11/2025  02:45p. m.  <DIR>          ..
01/11/2025  10:50p. m.               2.291 Card.class
01/11/2025  10:49p. m.               678 ClashRoyaleException.class
02/11/2025  01:39p. m.               3.833 Deck.class
01/11/2025  10:49p. m.               689 Thing.class
                                4 archivos          7.491 bytes
                                2 dirs  81.831.518.208 bytes libres
```

```
Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1\bin\test

02/11/2025  02:58p. m.  <DIR>          .
02/11/2025  02:45p. m.  <DIR>          ..
02/11/2025  01:35p. m.               6.671 DeckTest.class
                                1 archivos          6.671 bytes
                                2 dirs  81.821.757.440 bytes libres
```

```
C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1\bin\test>
```

```
C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1\bin\presentation>dir
El volumen de la unidad C es Acer
El número de serie del volumen es: 58EF-CB8D

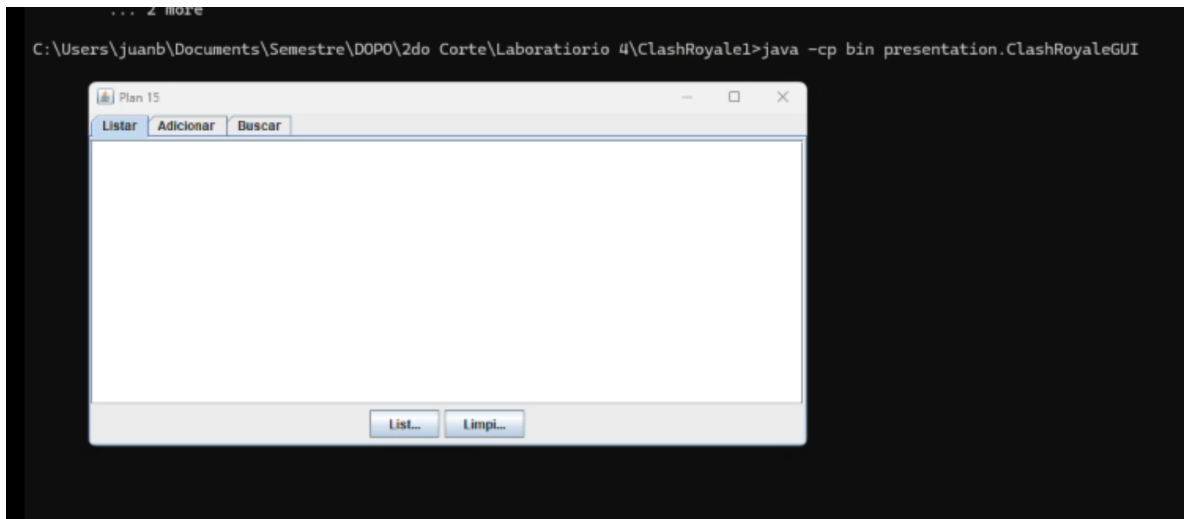
Directorio de C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1\bin\presentation

02/11/2025  03:08p. m.  <DIR>          .
02/11/2025  02:45p. m.  <DIR>          ..
02/11/2025  03:08p. m.               830 ClashRoyaleGUI$1.class
02/11/2025  03:08p. m.               795 ClashRoyaleGUI$2.class
02/11/2025  03:08p. m.               904 ClashRoyaleGUI$3.class
02/11/2025  03:08p. m.               794 ClashRoyaleGUI$4.class
02/11/2025  03:08p. m.               1.080 ClashRoyaleGUI$5.class
02/11/2025  03:08p. m.               996 ClashRoyaleGUI$6.class
02/11/2025  03:08p. m.               5.901 ClashRoyaleGUI.class
                                7 archivos          11.300 bytes
                                2 dirs  81.826.791.424 bytes libres

C:\Users\juanb\Documents\Semestre\DOPO\2do Corte\Laboratorio 4\ClashRoyale1\bin\presentation>
```

¿qué instrucciones debe dar para ejecutarlo?

1. Se ejecuta la clase principal: `java -cp bin presentation.ClashRoyaleGUI`

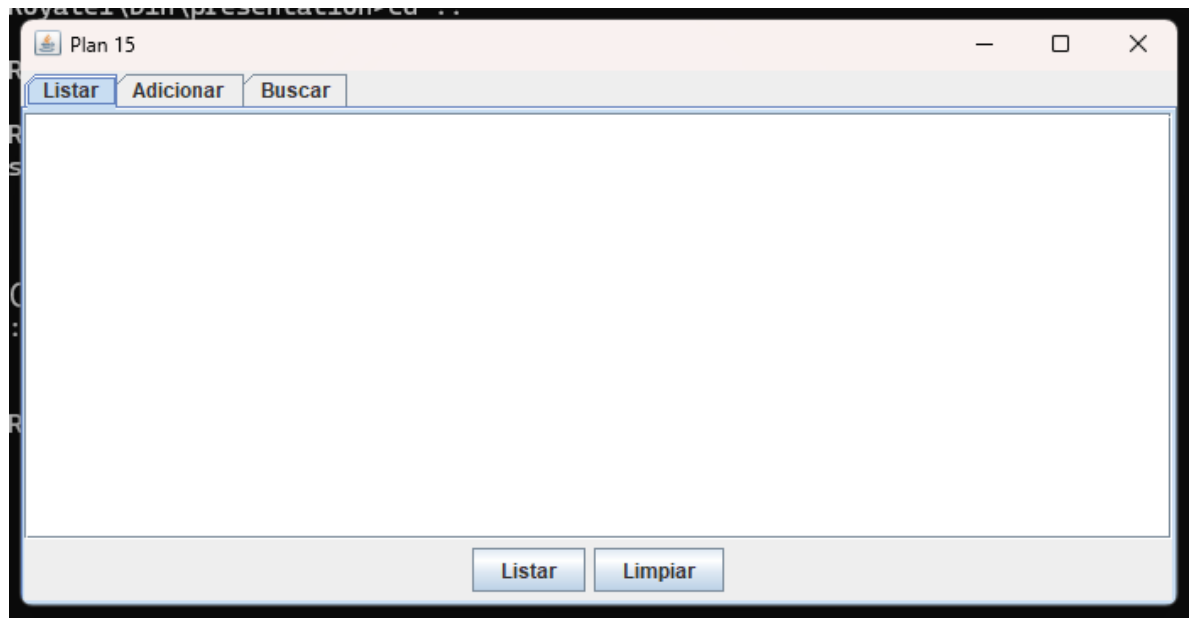


4. Ejecute el proyecto:

¿qué funcionalidades ofrece?

Ofrece, Listar, Adicionar y Buscar como aparece en la imagen:

En listar aparece la ventana en blanco.



En adicionar aparece las opciones de entrada de texto por teclado acompañado de etiquetas como Nombre, Tipo, Elixir(solo para cartas), Resistencia(solo para cartas), Cartas(solo para mazos).

Plan 15

Listar Adicionar Buscar

Nombre

Tipo

Elixir (solo para cartas)

Resistencia (solo para cartas)

Cartas (solo para mazos)

Adicionar Limpiar

En el apartado al buscar aparece la etiqueta buscar acompañado de la opción de entrada por texto.

Plan 15

Listar Adicionar Buscar

Busc...

¿cuáles funcionan?

En este caso probamos las tres funcionalidades que nos ofrecía (Listar, Adicionar, Buscar) pero ninguna de estas nos funcionó al probarlas.

5. Revisen el código y la documentación del proyecto.

¿De dónde salen los disfraces iniciales?

Salen del método addSome() de la clase ClashRoyale:

```
private void addSome(){
    String [][] cards= {{ "P.E.K.K.A", "TROOP", "7", "4982"},
                        {"Hog Rider", "TROOP", "4", "1696"},
                        {"Baby Dragon", "TROOP", "4", "1526"},
                        {"Cannon", "BUILDING", "3", "1176"},
                        {"Fireball", "SPELL", "4", "0"} };

    for (String [] c: cards){
        addCard(c[0],c[1],c[2],c[3]);
    }

    String [][] decks = {{ "Beatdown", "AGRESSIVE", "P.E.K.K.A\nHog Rider\nBaby Dragon"},
                        {"Control", "DEFENSIVE", "Baby Dragon\nCannon\nFireball"} };

    for (String [] c: decks){
        addDeck(c[0],c[1],c[2]);
    }
}
```

¿Qué clase pide que se adicionen?

La clase ClashRoyale.

```
/**
 * Add a new card
 */
public void addCard(String name, String type, String elixir, String hitPoints){
    Card nc=new Card(name,type,Integer.parseInt(elixir),Integer.parseInt(hitPoints));
    things.add(nc);
    cards.put(name.toUpperCase(),nc);
}

/**
 * Add a new deck
 */
public void addDeck(String name, String type, String theCards){
    Deck c = new Deck(name,type);
    String [] aCards= theCards.split("\n");
    for (String b : aCards){
        c.addCard(cards.get(b.toUpperCase()));
    }
    things.add(c);
}
```

¿Qué clase los adiciona?

La clase que los adiciona es Thing, se puede evidenciar por things.add en los códigos de arriba.

Adicionar y listar. Todo OK. [En lab04.doc, clashRoyale.asta y *.java]

(NO OLVIDEN BDD - MDD)

El objetivo es realizar ingeniería reversa a las funciones de adicionar y listar.

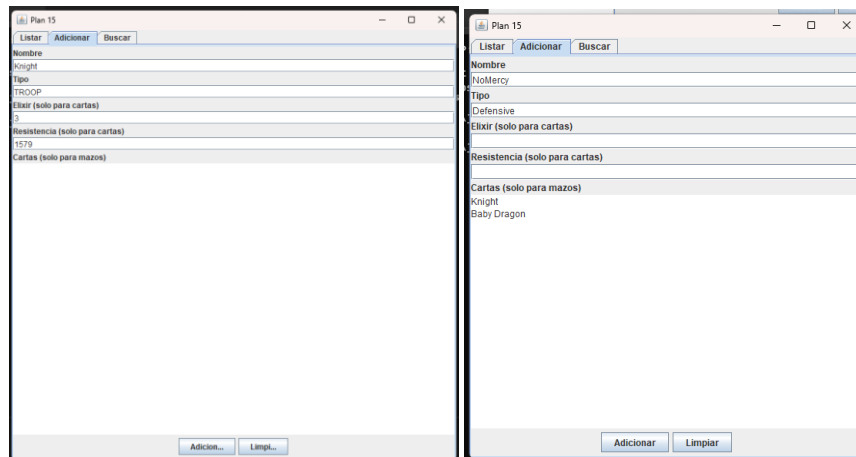
1. Adicionen una nueva carta y un nuevo mazo.

Card

Knight
Type: TROOP
Elixir: 3
Hitpoints: 1579

Mazo

NoMercy
Type: Defensive
Cards: Knight
Baby Dragon

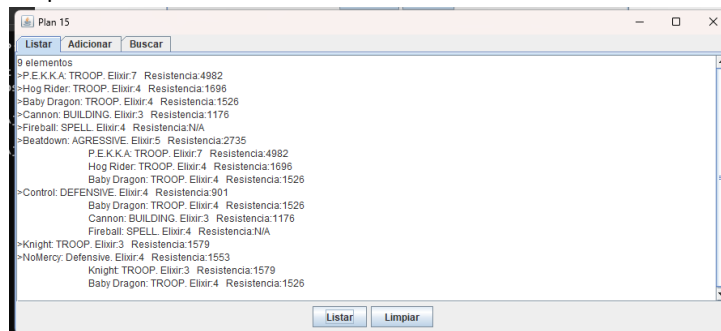


¿Qué ocurre?

Al momento de darle adicionar no ocurre nada, no se ve ni un mensaje de confirmación o de error, no hay evidencia de nada.

¿Cómo lo comprueban? Capturen la pantalla.

Se comprueba en la pestaña de lista dándole a listar.



¿Es adecuado este comportamiento?

Si, es adecuado se evidencia toda la lista de lo que se agregó.

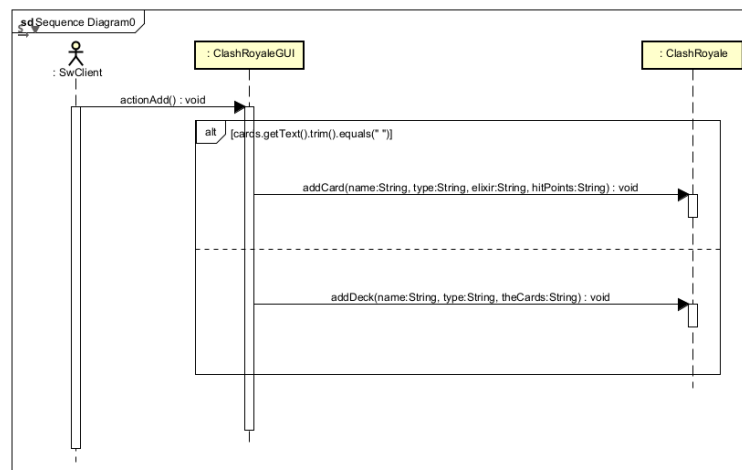
2. Revisen el código asociado a **adicionar** en la capa de presentación y la capa de dominio.

¿Qué método es responsable en la capa de presentación?

El método que es responsable de adicionar en la capa presentación es `actionAdd()`, se comprueba con el código del botón que usa ese método.

```
/*Add*/
buttonAdd.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent ev){
        actionAdd();
    }
});

private void actionAdd(){
    if (cards.getText().trim().equals("")){
        clash.addCard(name.getText(),type.getText(),elixir.getText(),hitPoints.getText());
    }else{
        clash.addDeck(name.getText(),type.getText(), cards.getText());
    }
}
```

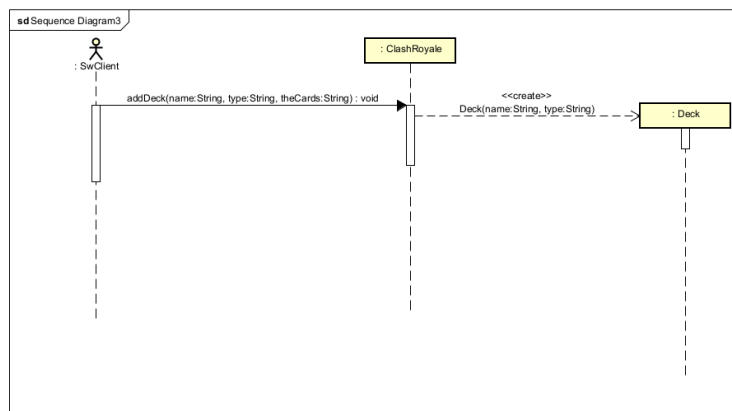
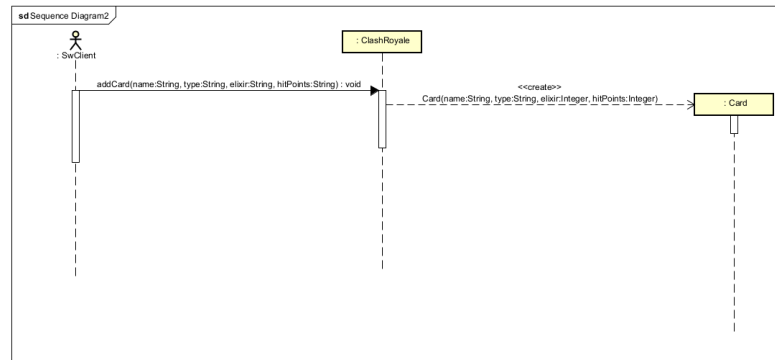


¿Qué método en la capa de dominio?

Los métodos que son responsables de adicionar en la capa de domain, son dos, el método que añade cartas `addCard()` y el método que añade el mazo `addDeck()`.

```
/**
 * Add a new card
 */
public void addCard(String name, String type, String elixir, String hitPoints){
    Card nc=new Card(name,type,Integer.parseInt(elixir),Integer.parseInt(hitPoints));
    things.add(nc);
    cards.put(name.toUpperCase(),nc);
}

/**
 * Add a new deck
 */
public void addDeck(String name, String type, String theCards){
    Deck c = new Deck(name,type);
    String [] aCards= theCards.split("\n");
    for (String b : aCards){
        c.addCard(cards.get(b.toUpperCase()));
    }
    things.add(c);
}
```



3. Realicen ingeniería reversa para la capa de dominio para **añadir**. Capturen los resultados de las pruebas de unidad.

```

@Test
public void shouldAddCardSuccessfully() {
    ClashRoyale c = new ClashRoyale();
    String name = "Giant";
    String type = "TROOP";
    String elixir = "5";
    String hitPoints = "3000";
    c.addCard(name, type, elixir, hitPoints);
    Thing card = c.consult("Giant");
    assertNotNull("La carta debería existir después de añadirla", card);
    assertEquals("El nombre de la carta debería ser Giant", "Giant", card.name());
    assertTrue("La carta debería ser una instancia de Card", card instanceof Card);
}
  
```

```

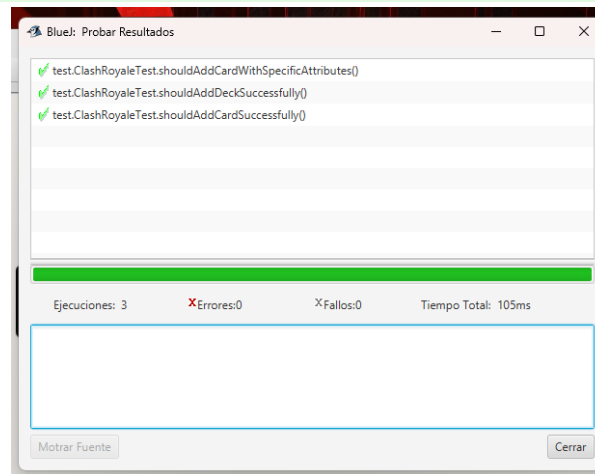
@Test
public void shouldAddDeckSuccessfully() {
    ClashRoyale c = new ClashRoyale();
    String deckName = "Test Deck";
    String deckType = "AGRESSIVE";
    String cards = "P.E.K.K.A\nHog Rider\nBaby Dragon";
    c.addDeck(deckName, deckType, cards);
    Thing deck = c.consult("Test Deck");
    assertNotNull("El mazo debería existir después de añadirlo", deck);
    assertEquals("El nombre del mazo debería ser Test Deck", "Test Deck", deck.name());
    assertTrue("El mazo debería ser una instancia de Deck", deck instanceof Deck);
}
  
```



```

@Test
public void shouldAddCardWithSpecificAttributes() throws ClashRoyaleException {
    ClashRoyale c = new ClashRoyale();
    String name = "Wizard";
    String type = "TROOP";
    String elixir = "5";
    String hitPoints = "2000";
    c.addCard(name, type, elixir, hitPoints);
    Thing card = c.consult("Wizard");
    assertNotNull("La carta debería existir", card);
    Card wizardCard = (Card) card;
    assertEquals("El tipo de carta debería ser TROOP", "TROOP", wizardCard.type());
    assertEquals("El costo de elixir debería ser 5", 5, wizardCard.elixir());
    assertEquals("Los puntos de vida deberían ser 2000", 2000, wizardCard.resistance());
}

```



- Revisen el código asociado a **listar** en la capa de presentación y la capa de dominio.

¿Qué método es responsable en la capa de presentación?

El método que es responsable de listar en la capa presentación es `actionList()`, se comprueba con el código del botón que usa ese método.

```

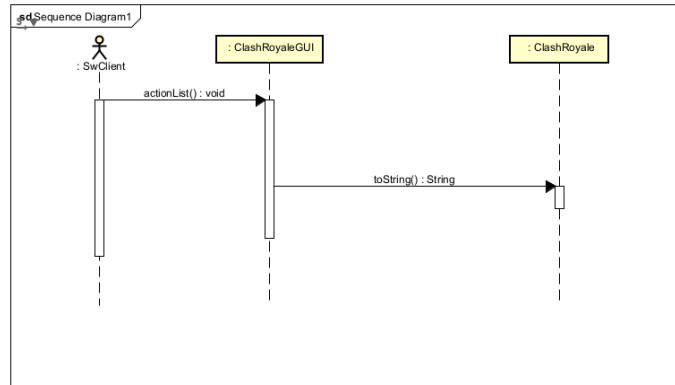
private void actionList(){
    textDetails.setText(clash.toString());
}

```

```

/*List*/
buttonList.addActionListener(new ActionListener(){
    public void actionPerformed(ActionEvent ev){
        actionList();
    }
});

```



¿Qué método en la capa de dominio?

El método que es responsable de listar en la capa dominio es `toString()`, se comprueba revisando el caso anterior, que retorna la lista de things.

```

/**
 * Return the data of all things
 * @return
 */
public String toString(){
    return data(things);
}
  
```

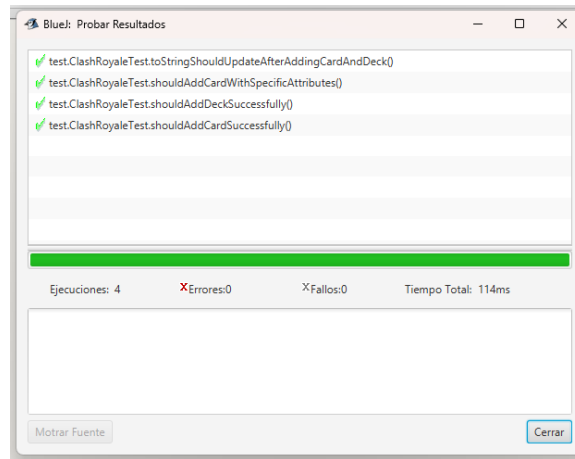
- Realicen ingeniería reversa para la capa de dominio para **listar**. Capturen los resultados de las pruebas de unidad.

```

@Test
public void shouldAddCardWithSpecificAttributes() throws ClashRoyaleException {
    ClashRoyale c = new ClashRoyale();
    String name = "Wizard";
    String type = "TROOP";
    String elixir = "5";
    String hitPoints = "2000";
    c.addCard(name, type, elixir, hitPoints);
    Thing card = c.consult("Wizard");
    assertNotNull("La carta debería existir", card);
    Card wizardCard = (Card) card;
    assertEquals("El tipo de carta debería ser TROOP", "TROOP", wizardCard.type());
    assertEquals("El costo de elixir debería ser 5", 5, wizardCard.elixir());
    assertEquals("Los puntos de vida deberían ser 2000", 2000, wizardCard.resistance());
}
  
```

```

@Test
public void toStringShouldUpdateAfterAddingCardAndDeck() {
    ClashRoyale c = new ClashRoyale();
    c.addCard("Giant", "TROOP", "5", "3000");
    c.addDeck("Test Deck", "AGRESSIVE", "P.E.K.K.A\nHog Rider\nBaby Dragon");
    String s = c.toString();
    assertTrue("Después de añadir una carta y un mazo el total debe incrementarse", s.startsWith("9 elementos"));
    assertTrue("Debe contener la carta añadida", s.contains("Giant"));
    assertTrue("Debe contener el mazo añadido", s.contains("Test Deck"));
}
  
```



6. Propongan y ejecuten una prueba de aceptación.

Verifica la creación de nuevas cartas, de un nuevo mazo usando esas cartas, comprueba el estado del juego, y verifica que se puede consultar los nuevos elementos añadidos.

```
@Test
public void shouldHandleCompleteGameScenario() {
    ClashRoyale c = new ClashRoyale();

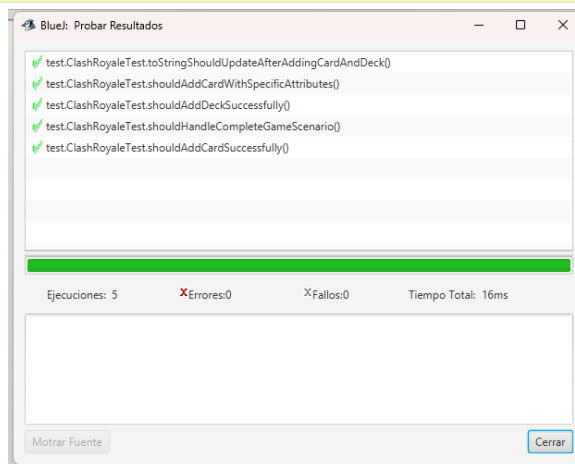
    c.addCard("Mega Knight", "TROOP", "7", "5000");
    c.addCard("Tesla", "BUILDING", "4", "1200");
    c.addDeck("Mixed Deck", "BALANCED", "Mega Knight\nTesla\nFireball");

    String gameState = c.toString();

    assertTrue("El juego debe tener las nuevas cartas y el mazo", gameState.contains("Mega Knight") && gameState.contains("Tesla") &&
        gameState.contains("Mixed Deck"));

    Thing megaKnight = c.consult("Mega Knight");
    Thing mixedDeck = c.consult("Mixed Deck");

    assertNotNull("Debe existir la carta Mega Knight", megaKnight);
    assertNotNull("Debe existir el mazo Mixed Deck", mixedDeck);
    assertTrue("El número total de elementos debe ser correcto",
        gameState.startsWith("10 elementos"));
}
```



Adicionar una carta o un mazo. Funcionalidad robusta [En lab04.doc, clashRoyale.asta y *.java]

(NO OLVIDEN BDD – MDD)

El objetivo es perfeccionar la funcionalidad de adicionar una carta para hacerla más robusta.

Para cada uno de los siguientes casos realice los pasos del 1 al 4.

- a. ¿Y si el nombre de la carta o el mazo ya existe?
 - b. ¿Y si los valores numéricos no son numéricos?
 - c. ¿Y si el elixir no tiene los valores esperados?
 - d. Proponga una nueva condición
1. Propongan una prueba de aceptación que genere el fallo.

```
// a
/**
 * Si el nombre de la carta o el mazo ya existe, debe fallar.
 */
@Test
public void shouldFailWhenCardNameOrDeckAlreadyExists() {
    ClashRoyale c = new ClashRoyale();
    try {
        c.addCard("C001", "Gigante", "5", "800");
        c.addCard("C002", "Gigante", "5", "800");
        fail("Debería lanzar una excepción por carta duplicada.");
    } catch (Exception e) {
        assertEquals(ClashRoyaleException.DUPLICATE_CARD, e.getMessage());
    }
}
```

```
// b
/**
 * Si los valores numéricos no son válidos, debe fallar.
 */
@Test
public void shouldFailWhenNumericValuesAreInvalid() {
    ClashRoyale c = new ClashRoyale();
    try {
        c.addCard("C010", "Mago", "cinco", "ochocientos");
        fail("Debería lanzar una excepción por valores no numéricos.");
    } catch (Exception e) {
        assertEquals(ClashRoyaleException.INVALID_NUMBER, e.getMessage());
    }
}
```

```

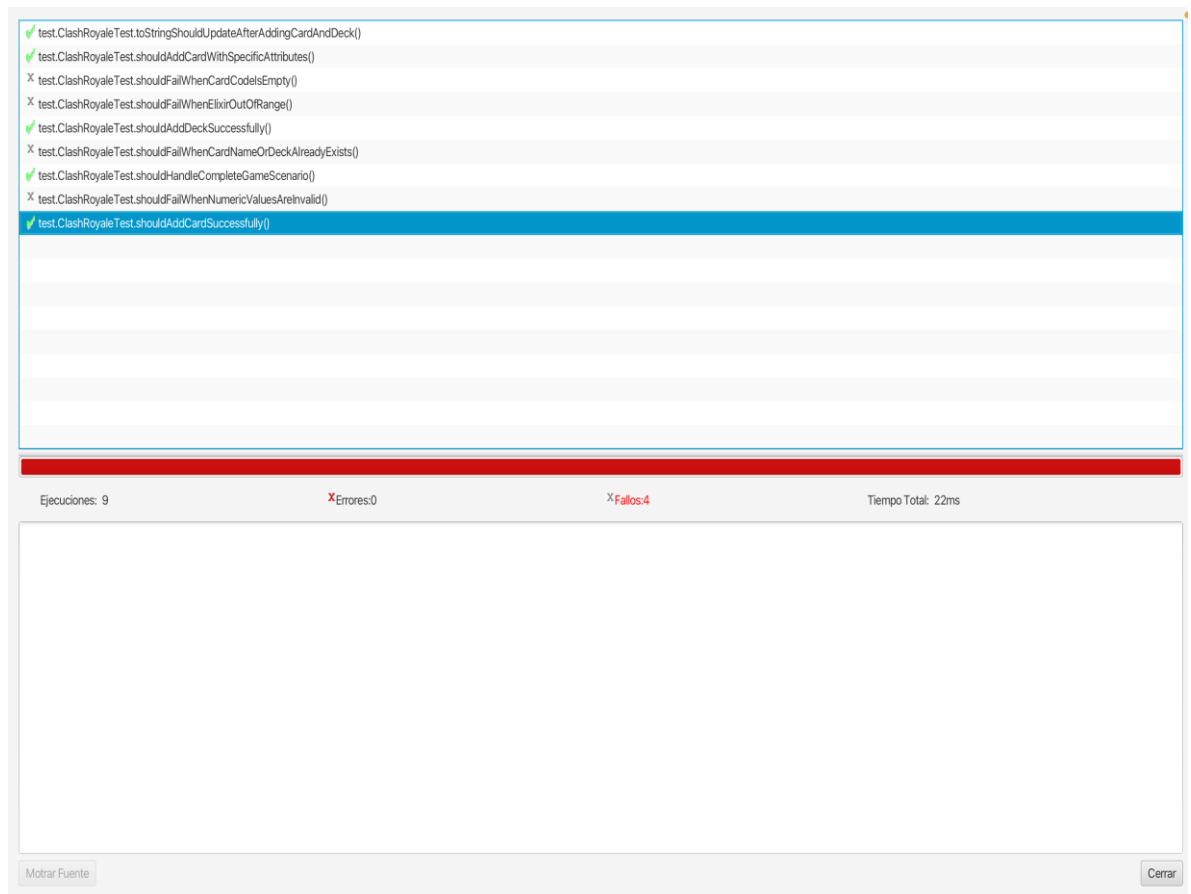
// c
/**
 * Si el elixir está fuera del rango permitido, debe fallar.
 */
@Test
public void shouldFailWhenElixirOutOfRange() {
    ClashRoyale c = new ClashRoyale();

    try {
        c.addCard("C020", "Mini P.E.K.K.A", "20", "600");
        fail("Debería lanzar una excepción por elixir fuera de rango.");
    } catch (Exception e) {
        assertEquals(ClashRoyaleException.ELIXIR_BAD_ERROR, e.getMessage());
    }
}

// d
/**
 * Si el código de la carta está vacío, debe fallar.
 */
@Test
public void shouldFailWhenCardCodeIsEmpty() {
    ClashRoyale c = new ClashRoyale();

    try {
        c.addCard("", "Arqueras", "3", "200");
        fail("Debería lanzar una excepción por código vacío.");
    } catch (Exception e) {
        assertEquals(ClashRoyaleException.INVALID_TYPE, e.getMessage());
    }
}

```



Propusimos uno de código repetido para que nos arroje el fallo, y lo que está luego del test, quiere decir que el funcionamiento espera una exception de ClashRoyaleException si no, la prueba falla.

- Analicen el diseño realizado. Para hacer el software robusto:

¿Qué método debería lanzar la excepción?

Consideramos que el método que debería lanzar la excepción es addCard, porque es el que recibe los datos como String, realiza las validaciones y conversiones necesarias (por ejemplo, verificar si los valores son numéricos o si la carta ya existe). Por lo tanto, este método es responsable de detectar y lanzar la excepción cuando algo no cumple las condiciones esperadas.

¿Qué métodos deberían propagarla?

Los métodos que deberían propagar la excepción son aquellos que invocan a addCard, como los métodos de la capa de presentación o control. Estos métodos no deben manejar directamente la validación interna, sino propagar la excepción si no la pueden resolver, para que sea atendida en un nivel superior.

¿Qué método debería atenderla? Explique claramente.

- La capa de presentación o interfaz de usuario debería ser la encargada de atender la excepción. Es decir, debe:
 - Mostrar un mensaje claro al usuario indicando el error.
 - Evitar que la aplicación se detenga o colapse.
 - Registrar o reportar el error si es necesario para su análisis posterior.

3. Construya la solución propuesta. Capture los resultados de las pruebas de unidad.

```
/**
 * Adds a new card to the collection.
 *
 * @param name Unique name of the card.
 * @param type Type of the card (e.g., TROOP, BUILDING, SPELL).
 * @param elixir Amount of elixir required to use the card, given as a string.
 * @param hitPoints Hit points of the card, given as a string.
 * @throws ClashRoyaleException If the name is empty, numeric values are invalid, or a card with the same name already exists.
 */
public void addCard(String name, String type, String elixir, String hitPoints) throws ClashRoyaleException {
    if (name == null || name.trim().isEmpty()) {
        throw new ClashRoyaleException(ClashRoyaleException.INVALID_TYPE);
    }
    if (type == null || type.trim().isEmpty()) {
        throw new ClashRoyaleException(ClashRoyaleException.INVALID_TYPE);
    }
    int elixirValue;
    int hitPointsValue;
    try {
        elixirValue = Integer.parseInt(elixir);
        hitPointsValue = Integer.parseInt(hitPoints);
    } catch (NumberFormatException e) {
        throw new ClashRoyaleException(ClashRoyaleException.INVALID_NUMBER);
    }
    if (elixirValue < 1 || elixirValue > 10) {
        throw new ClashRoyaleException(ClashRoyaleException.ELIXIR_BAD_ERROR);
    }
    if (cards.containsKey(name.toUpperCase())) {
        throw new ClashRoyaleException(ClashRoyaleException.DUPLICATE_CARD);
    }

    Card nc = new Card(name, type, elixirValue, hitPointsValue);
    things.add(nc);
    cards.put(name.toUpperCase(), nc);
}

/**
 * Loads a predefined set of sample cards and decks into the system.
 *
 * This method is used to initialize default data for testing or demonstration purposes.
 * If any card fails to be added (for example, due to invalid data),
 * the error is handled, and the process continues with the next item.
 *
 * @throws ClashRoyaleException If the method directly calls #addCard(String, String, String, String)
 * without handling exceptions internally.
 */
private void addSome() {
    String [][] cards = {
        {"P.E.K.K.A", "TROOP", "7", "4982"},
        {"Hog Rider", "TROOP", "4", "1696"},
        {"Baby Dragon", "TROOP", "4", "1526"},
        {"Cannon", "BUILDING", "3", "1176"},
        {"Fireball", "SPELL", "4", "0"}
    };

    for (String [] c : cards) {
        try {
            addCard(c[0], c[1], c[2], c[3]);
        } catch (ClashRoyaleException e) {
            System.out.println("No se pudo agregar la carta " + c[0] + ": " + e.getMessage());
        }
    }

    String [][] decks = {
        {"Beatdown", "AGRESSIVE", "P.E.K.K.A\nHog Rider\nBaby Dragon"},
        {"Control", "DEFENSIVE", "Baby Dragon\nCannon\nFireball"}
    };

    for (String [] c : decks) {
        addDeck(c[0], c[1], c[2]);
    }
}
```

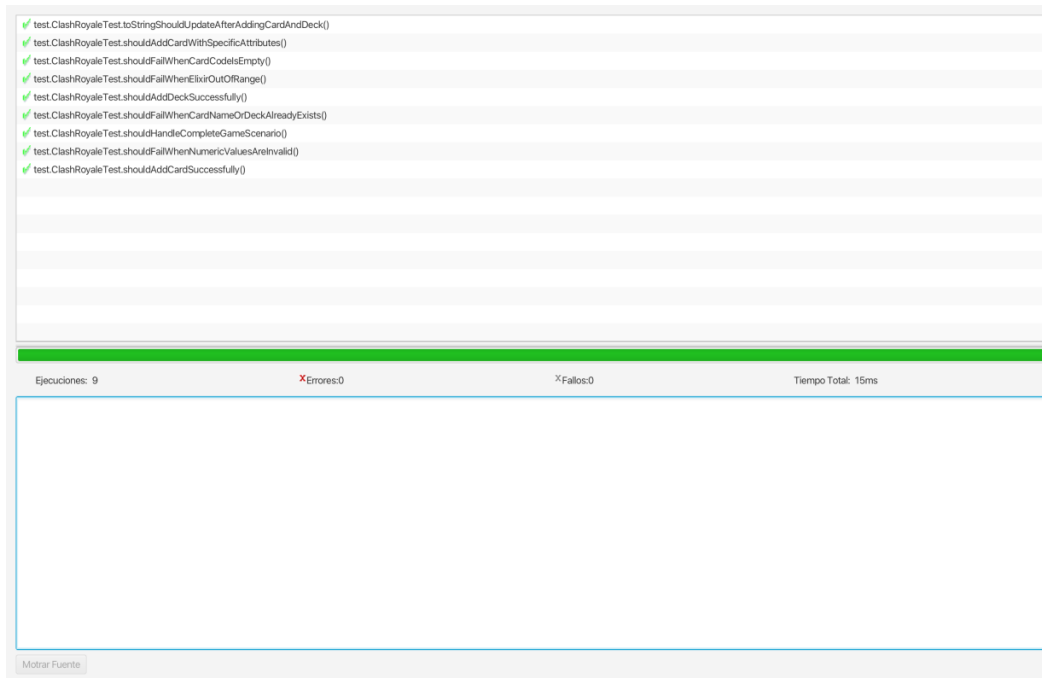
test.ClashRoyaleTest.toStringShouldUpdateAfterAddingCardAndDeck()
test.ClashRoyaleTest.shouldAddCardWithSpecificAttributes()
test.ClashRoyaleTest.shouldFailWhenCardCodeIsEmpty()
test.ClashRoyaleTest.shouldFailWhenElixirOutOfRange()
test.ClashRoyaleTest.shouldAddDeckSuccessfully()
test.ClashRoyaleTest.shouldFailWhenCardNameOrDeckAlreadyExists()
test.ClashRoyaleTest.shouldHandleCompleteGameScenario()
test.ClashRoyaleTest.shouldFailWhenNumericValuesAreInvalid()
test.ClashRoyaleTest.shouldAddCardSuccessfully()

Ejecuciones: 9 Errores: 0 Fallos: 0 Tiempo Total: 15ms

Mostrar Fuente

4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1. ¿Qué sucede ahora? Capture la pantalla.

```
public static void main(String args[]){  
    ClashRoyaleGUI gui=new ClashRoyaleGUI();  
    gui.setVisible(true);  
}
```

Pasan las pruebas de unidad propuestas en 1 que fallaban.

Consultando por patrones. ¡No funciona y queda sin funcionar!

[En clashRoyale.asta, clashRoyale.log, lab04.java y *.java]

(NO OLVIDEN BDD - MDD)

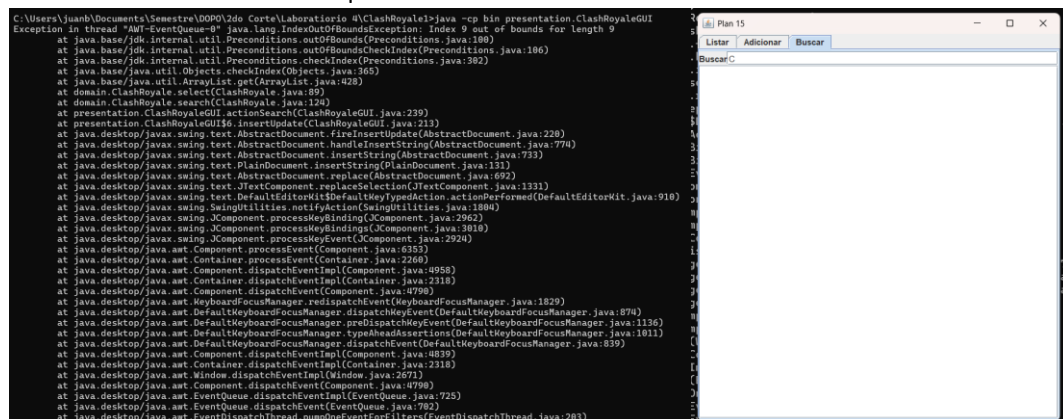
1. Consulten un mazo que inicie con C.

¿Qué sucede?

No da resultado y nos lanza una excepción.

¿Qué creen que pasó? Capturen el resultado.

Se intenta acceder a un índice que esta afuera de la lista.



¿Quién debe conocer y quien NO debe conocer esta información?

Solo los desarrolladores deben conocer esta información, ya que son los que deben solucionarlo y saben que pasa, en cambio los usuarios se asustan si ven ese error, no van a saber que hacer.

2. Exploren el método [record](#) de la clase [Log](#)

¿Qué servicio presta?

El método se encarga de registrar errores o las excepciones que ocurren cuando se ejecuta el programa, y los escribe en un archivo .log.

```

/**
 *
 */
public class Log{
    public static String name="ClashRoyale";

    public static void record(Exception e){
        try{
            Logger logger=Logger.getLogger(name);
            logger.setUseParentHandlers(false);
            FileHandler file=new FileHandler(name+".log",true);
            file.setFormatter(new SimpleFormatter());
            logger.addHandler(file);
            logger.log(Level.SEVERE,e.toString(),e);
            file.close();
        }catch (Exception oe){
            oe.printStackTrace();
            System.exit(0);
        }
    }
}

```

- Analicen el punto adecuado para que **EN ESTE CASO** se presente un mensaje especial de alerta al usuario, se guarde la información del error en el registro y continúe la ejecución.

Expliquen y construyan la solución.

Nuestra solución es que la acción de buscar atienda la excepción y después construimos un método que la muestre y la grabe en la clase log.

```

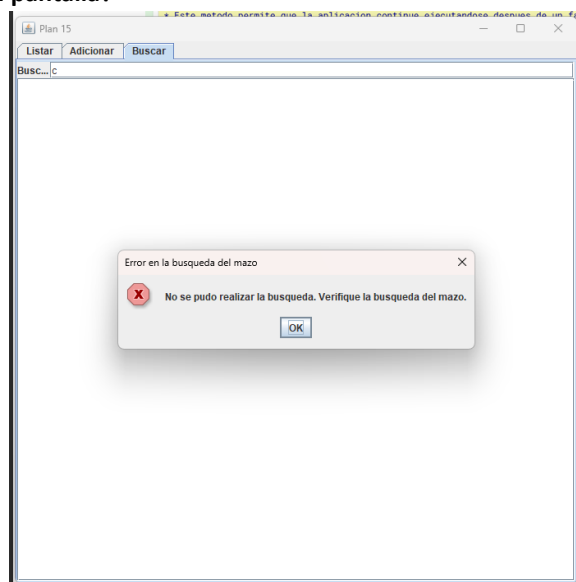
/**
 * Realiza la acción de búsqueda.
 */
private void actionSearch(){
    String patronBusqueda = textSearch.getText().trim();
    String answer = "";
    if (patronBusqueda.length() > 0) {
        try{
            answer = clash.search(patronBusqueda);
        } catch (Exception e){
            manejarError(e, "No se pudo realizar la búsqueda. Verifique la búsqueda del mazo.");
        }
    }
    textResults.setText(answer);
}

private void manejarError(Exception e, String mensajeUsuario){
    JOptionPane.showMessageDialog(this, mensajeUsuario, "Error en la búsqueda del mazo", JOptionPane.ERROR_MESSAGE);
    Log.record(e);
}

```

- Ejecuten nuevamente la aplicación con el caso propuesto en 1.

¿Qué mensaje salió en pantalla?



¿La aplicación termina?

No, no termina, si le damos aceptar, sigue funcionando.

¿Qué información tiene el archivo de errores?

```
nov 02, 2025 8:36:14 P. M. domain.Log record
SEVERE: java.lang.IndexOutOfBoundsException: Index 7 out of bounds for length 7
java.lang.IndexOutOfBoundsException: Index 7 out of bounds for length 7
    at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:100)
    at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Preconditions.java:106)
    at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:302)
    at java.base/java.util.Objects.checkIndex(Objects.java:365)
    at java.base/java.util.ArrayList.get(ArrayList.java:428)
    at domain.ClashRoyale.select(ClashRoyale.java:87)
    at domain.ClashRoyale.search(ClashRoyale.java:120)
    at presentation.ClashRoyaleGUI.actionSearch(ClashRoyaleGUI.java:243)
    at presentation.ClashRoyaleGUI$6.insertUpdate(ClashRoyaleGUI.java:213)
    at java.desktop/javafx.swing.text.AbstractDocument.fireInsertUpdate(AbstractDocument.java:220)
    at java.desktop/javafx.swing.text.AbstractDocument.handleInsertString(AbstractDocument.java:774)
    at java.desktop/javafx.swing.text.AbstractDocument.insertString(AbstractDocument.java:733)
    at java.desktop/javafx.swing.text.PlainDocument.insertString(PlainDocument.java:131)
    at java.desktop/javafx.swing.text.AbstractDocument.replace(AbstractDocument.java:692)
    at java.desktop/javafx.swing.text.JTextComponent.replaceSelection(JTextComponent.java:1331)
    at java.desktop/javafx.swing.text.DefaultEditorKit$DefaultKeyTypedAction.actionPerformed(DefaultEditorKit.java:910)
    at java.desktop/javafx.swing.SwingUtilities.notifyAction(SwingUtilities.java:1804)
    at java.desktop/javafx.swing.JComponent.processKeyBinding(JComponent.java:2962)
    at java.desktop/javafx.swing.JComponent.processKeyBindings(JComponent.java:3010)
    at java.desktop/javafx.swing.JComponent.processKeyEvent(JComponent.java:2924)
    at java.desktop/java.awt.Component.processEvent(Component.java:6353)
    at java.desktop/java.awt.Container.processEvent(Container.java:2260)
    at java.desktop/java.awt.Component.dispatchEventImpl(Component.java:4958)
    at java.desktop/java.awt.Container.dispatchEventImpl(Container.java:2318)
    at java.desktop/java.awt.Component.dispatchEvent(Component.java:4790)
    at java.desktop/java.awt.KeyboardFocusManager.redispatchEvent(KeyboardFocusManager.java:1829)
    at java.desktop/java.awt.DefaultKeyboardFocusManager.dispatchEvent(DefaultKeyboardFocusManager.java:874)
    at java.desktop/java.awt.DefaultKeyboardFocusManager.preDispatchKeyEvent(DefaultKeyboardFocusManager.java:1136)
    at java.desktop/java.awt.DefaultKeyboardFocusManager.typeAheadAssertions(DefaultKeyboardFocusManager.java:1011)
    at java.desktop/java.awt.DefaultKeyboardFocusManager.dispatchEvent(DefaultKeyboardFocusManager.java:839)
    at java.desktop/java.awt.Component.dispatchEventImpl(Component.java:4839)
    at java.desktop/java.awt.Container.dispatchEventImpl(Container.java:2318)
    at java.desktop/java.awt.Window.dispatchEventImpl(Window.java:2671)
    at java.desktop/java.awt.Component.dispatchEvent(Component.java:4790)
    at java.desktop/java.awt.EventQueue.dispatchEventImpl(EventQueue.java:725)
    at java.desktop/java.awt.EventQueue.dispatchEvent(EventQueue.java:702)
    at java.desktop/java.awt.EventDispatchThread.pumpOneEventForFilters(EventDispatchThread.java:203)
    at java.desktop/java.awt.EventDispatchThread.pumpEventsForFilter(EventDispatchThread.java:124)
    at java.desktop/java.awt.EventDispatchThread.pumpEventsForHierarchy(EventDispatchThread.java:113)
    at java.desktop/java.awt.EventDispatchThread.pumpEvents(EventDispatchThread.java:109)
    at java.desktop/java.awt.EventDispatchThread.pumpEvents(EventDispatchThread.java:101)
    at java.desktop/java.awt.EventDispatchThread.run(EventDispatchThread.java:90)
```

5. ¿Es adecuado que la aplicación continúe su ejecución después de sufrir un incidente como este?

Como el error solo afecta una búsqueda y no afecta todo el resto del programa puede seguir funcionando.

¿De qué dependería continuar o parar?

Depende de la gravedad del error, si es leve como el de la búsqueda puede continuar, pero si es un error grave como una pérdida de datos, errores de memoria debe parar y revisarse.

6. Modifiquen la aplicación para garantizar que SIEMPRE que haya un error se maneje de forma adecuada.

¿Cuál fue la solución implementada?

```
private void addSome(){
    try{
        String [][] cards= {{ "P.E.K.K.A", "TROOP","7","4982"},
                               {"Hog Rider", "TROOP","4","1696"},
                               {"Baby Dragon", "TROOP","4","1526"},
                               {"Cannon", "BUILDING","3","1176"},
                               {"Fireball", "SPELL","4","0"} };

        for (String [] c: cards){
            addCard(c[0],c[1],c[2],c[3]);
        }

        String [][] decks = {{ "Beatdown","AGGRESSIVE", "P.E.K.K.A\nHog Rider\nBaby Dragon"},
                               {"Control","DEFENSIVE", "Baby Dragon\nCannon\nFireball"} };

        for (String [] c: decks){
            addDeck(c[0],c[1],c[2]);
        }

    } catch (ClashRoyaleException e){
        JOptionPane.showMessageDialog(null, "Error al cargar los datos:\n" + e.getMessage(), "Error de carga", JOptionPane.ERROR_MESSAGE);
        Log.record(e);
    }
}
```

Consultando por patrones. ¡Ahora si funciona!

[En clashRoyale.asta, clashRoyale.log, lab04.java y *.java]

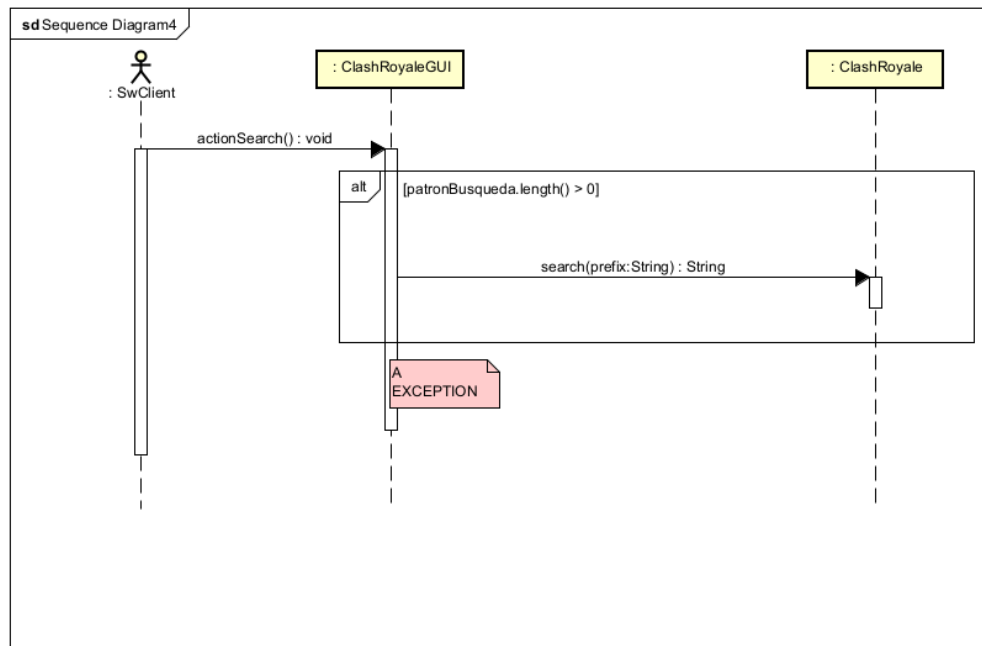
(NO OLVIDEN BDD - MDD)

1. Revisen el código asociado a **buscar** en la capa de presentación y la capa de dominio.

¿Qué método es responsable en la capa de presentación?

El método responsable en presentación es actionSearch()

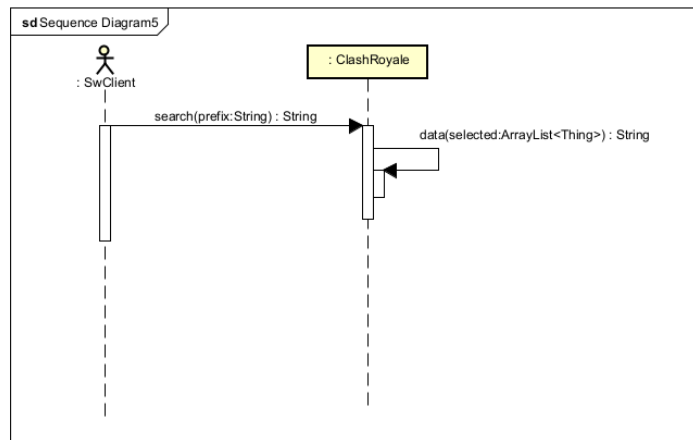
```
/**
 * Realiza la acción de búsqueda.
 */
private void actionSearch(){
    String patronBusqueda = textSearch.getText().trim();
    String answer = "";
    if (patronBusqueda.length() > 0) {
        try{
            answer = clash.search(patronBusqueda);
        } catch (Exception e){
            manejarError(e, "No se pudo realizar la búsqueda. Verifique la búsqueda del mazo.");
        }
    }
    textResults.setText(answer);
}
```



¿Qué método es responsable en la capa de dominio?

El método responsable en dominio es Search()

```
/**
 * Return the data of things with a prefix
 * @param prefix
 * @return
 */
public String search(String prefix){
    return data(select(prefix));
}
```

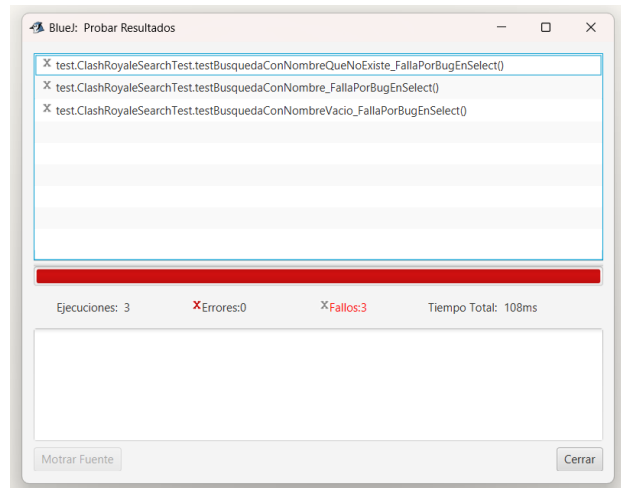


2. Realicen ingeniería reversa para la capa de dominio para buscar. Capturen los resultados de las pruebas. Deben fallar.

```
public class ClashRoyaleSearchTest
{
    @Test
    public void testBusquedaConNombre_FallaPorBugEnSelect() throws ClashRoyaleException {
        ClashRoyale c = new ClashRoyale();
        assertDoesNotThrow(() -> {
            c.search("Cannon");
        });
    }

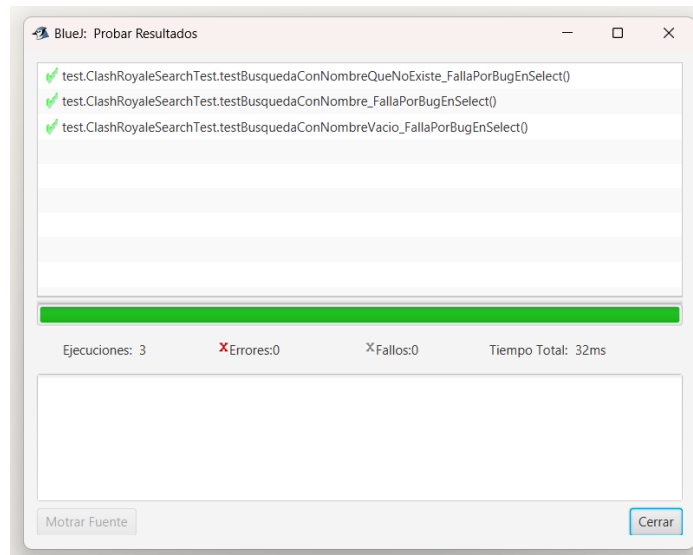
    @Test
    public void testBusquedaConNombreVacio_FallaPorBugEnSelect() throws ClashRoyaleException {
        ClashRoyale c = new ClashRoyale();
        assertDoesNotThrow(() -> {
            c.search("");
        });
    }

    @Test
    public void testBusquedaConNombreQueNoExiste_FallaPorBugEnSelect() throws ClashRoyaleException {
        ClashRoyale c = new ClashRoyale();
        assertDoesNotThrow(() -> {
            c.search("Messi");
        });
    }
}
```



3. ¿Cuál es el error? Solúcionenlo. Capturen los resultados de las pruebas.
El error está en el método select.

```
/**
 * Consults the things that start with a prefix
 * @param
 * @return
 */
public ArrayList<Thing> select(String prefix){
    ArrayList <Thing> answers=new ArrayList<Thing>();
    prefix=prefix.toUpperCase();
    for(int i=0;i<things.size();i++){
        if(things.get(i).name().toUpperCase().startsWith(prefix)){
            answers.add(things.get(i));
        }
    }
    return answers;
}
```

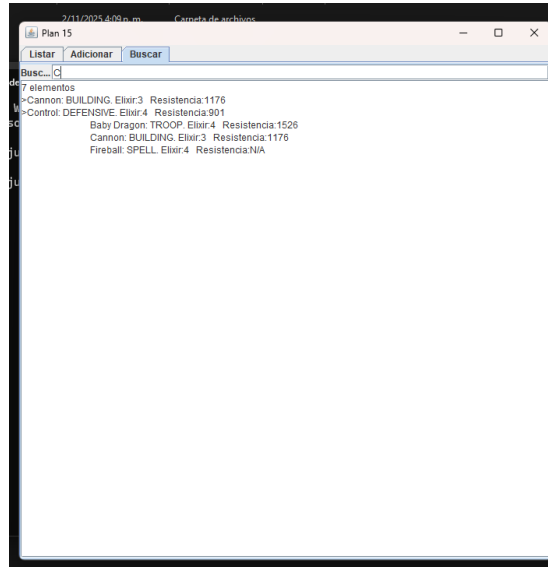


4. Ejecuten la aplicación nuevamente con el caso propuesto.

¿Qué tenemos en pantalla?

¿Qué información tiene el archivo de errores?

Se nos muestra lo que estamos buscando, el archivo de errores, no se actualizó, no muestra nada nuevo.



5. Refactorice la funcionalidad para que sea más amable con el usuario.

¿Cuál es la propuesta?

La propuesta sería avisar cuando no exista un resultado en la búsqueda.

¿Cómo la implementa?

```
/**
 * Realiza la acción de búsqueda.
 */
private void actionSearch(){
    String patronBusqueda = textSearch.getText().trim();
    String answer = "";
    if (patronBusqueda.length() > 0) {
        try{
            answer = clash.search(patronBusqueda);
            if(answer.trim().isEmpty() || answer.contains("No existe el mazo/carta")) {
                answer = "No se encontraron cartas o mazos con la letra indicada. ";
            }
        } catch (Exception e){
            manejarError(e, "No se pudo realizar la búsqueda. Verifique la búsqueda del mazo.");
        }
    }
    textResults.setText(answer);
}
```

RETROSPECTIVA

- 1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)**
Bernal = 15 horas
Bogotá = 15 horas
- 2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?**
Completo porque logramos acabar todo y entender cada punto del laboratorio.
- 3. Considerando las prácticas XP del laboratorio. ¿cuál fue la más útil? ¿Por qué?**
Dos programadores trabajan juntos en el código (Pair Programming). Esta es la práctica XP que nos fue más útil a la hora de realizar este laboratorio, puesto que nosotros nos intercambiábamos la labor de escribir código y revisar el código de la otra persona. También porque intercambiamos ideas y conocimiento que fuimos adquiriendo en el auto estudio de cada persona.
- 4. ¿Cuál consideran fue el mayor logro? ¿Por qué?**
Entender todos los requerimientos que exigía el laboratorio a partir de las preguntas que fueron expuestas para su resolución. No dejando por nuestra parte lugar a la ambigüedad de nuestras respuestas.
- 5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?**
El mayor problema técnico que presentamos a la hora de realizar este laboratorio fue la no interconectividad que presenta Blue J, donde no podíamos escribir en el mismo archivo al mismo tiempo, lo cual nos llevó a buscar otro entorno virtual que satisficiera esa necesidad. Luego de esto volver a Blue J y generar así el "Shapes" final.
- 6. ¿Qué hicieron bien como actividades? ¿Qué se comprometen a hacer para mejorar los resultados?**
Repartir de una buena forma el trabajo para que ambos aprendamos y además tengamos la misma carga de trabajo. Nos comprometemos a repartir mejor las horas por día para hacer el laboratorio.
- 7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.**
ChatGPT. (s/f). Chatgpt.com. Recuperado el 20 de febrero de 2025, de <https://chatgpt.com/>
Rodríguez, A. (s. f.). Análisis y casos. Grafos - software para la construcción, edición y análisis de grafos. Recuperado el 22 de febrero de 2025, de <https://arodrigu.webs.upv.es/grafos/doku.php?id=analisis>
Vypirailenko, A. (2023, julio 21). División entera Java. CodeGym. <https://codegym.cc/es/groups/posts/es.696.division-entera-java>
El mejor fue Análisis y casos, ya que una de las cosas que más nos tomó tiempo, fue conocer cómo debía funcionar la calculadora de grafos, esa página nos ayudó mucho con la parte teórica para su implementación.